[code_analysis] Semantic check - Checking Semantic of Particular Statement

Created By	Hyung-Gyu Ryoo
Email	hgryoo@cubrid.com
Tags	코드분석
Team	개발2팀
Version	0.8

Checking Semantic of Particular Statement Overview
Semantic checks aggregate/analytic function on where clause
Select list 관련 검사
- illegal multi-column subquery
- INTO
- Click counter

GROUP BY (Aggreagte/Analytic 함수) 관련
- 범위를 벗어난 GROUP BY position 에러
- host variable를 그룹 조건으로 입력 불가
- WITH ROLLUP과 GROUPBY_NUM을 동시에 사용 못함
- Aggregate/Analytic 함수의 인수 체크
- MEDIAN 함수와 OVER (order by...)
- PARTITION BY NULL 제거

ORDER BY 관련
- 필요 없는 ORDER BY를 제거
- position 의 범위가 select list 의 밖으로 나갔는지 확인

Hierarchical Query 관련 **(TBU)**
Show statement 관련
Derived query 관련
- Ambiguous reference 검사
- Hidden column rewrite

Expression 관련
- Check CAST expression is well-formed

LIMIT가 있는 경우 rewriting
- LIMIT가 있는 경우 Aggregate를 derived table로 rewriting
- LIMIT를 Numbering Expression으로 rewriting

Search Condition 관련

Checking Semantic of Particular Statement Overview

이 단계에서는 각 구문별로 다음을 수행한다.

- 성능과 관계없는 rewrite
- 의미적인 에러를 확인하고 에러를 반환

rewrite를 하는 경우는 의미적으로 불필요한 부분이거나 SQL 구문에서는 여러가지 방식으로 작성할 수 있지만 내부적으로는 하나의 방식으로만 처리하기 위해서 재작성한다. 어떤 경우에 rewrite가 수행되고 결과는 무엇인지 예시를 제시하는 수준으로 설명하려고 한다. 에러를 반환하는 경우에는 에러를 반환하는 예시와 함께 어떤 에러를 반환하는지만을 설명한다.

Semantic checks aggregate/analytic function on where clause

```
// ERROR ->
SELECT
        p.projectName,
        SUM(a.costtodate) AS sumActivityCosts,
        p.fundedbudget
    FROM Project AS p
    LEFT OUTER JOIN Activity a ON p.projectid = a.projectid
    WHERE
        a.activityId IS NULL
```

```
        AND p.projectid IS NOT NULL
        AND SUM(a.costtodate) > p.fundedbudget
    GROUP BY
        p.projectID
```

위 예시의 SUM 함수와 같이 aggregate나 analytic 함수는 where 절에 존재할 수 없다. 따라서 where 절에 이러한 함수가 존재하는지 파싱 트리에서 재귀적으로 확인하고, 만약 존재한다면 에러를 반환한다.

> MSGCAT_SEMANTIC_INVALID_AGGREGATE
> MSGCAT_SEMANTIC_NESTED_ANALYTIC_FUNCTIONS

참조 코드: pt_check_where()

Select list 관련 검사

illegal multi-column subquery

parser 단계에서 검사하지만 이 단계에서도 검사하는 로직이 구현되어 있다.

```
SELECT name, capital,
(SELECT host_city, slogan FROM olympic WHERE host_nation = name) AS host_cities
FROM nation;
```

> MSGCAT_SEMANTIC_NOT_SINGLE_COL

INTO

INTO에서 INTO 이전의 컬럼의 개수와 INTO 이후의 :identifier의 개수가 같아야 한다.

```
SELECT 'Hi', 'Hey' INTO :out_data FROM db_root;
```

> MSGCAT_SEMANTIC_COL_CNT_NE_INTO_CNT

INTO는 subquery에서 사용할 수 없다.

```
SELECT 'Hi', 'Hey' INTO :out_data FROM db_root;
```

> MSGCAT_SEMANTIC_SELECT_INTO_IN_SUBQUERY

코드 참조: pt_check_into_clause()

Click counter

WITH INCREMENT 구문이 있는 경우 다음과 같이 rewrite 한다.

```
SELECT content FROM board WHERE id=1 WITH INCREMENT FOR read_count;

=> rewritten query (including hidden column)

SELECT board.content,  incr(board.read_count)
FROM board board WHERE board.id= ?:0
```

GROUP BY (Aggreagte/Analytic 함수) 관련

GROUP BY

SELECT ...
GROUP BY **{col_name | expr | position}** [ASC | DESC], ...
[WITH ROLLUP] [HAVING <search_condition>]

GROUP BY 에는 컬럼 이름이나 expression, 그리고 select list 중 하나를 가리키는 position을 입력할 수 있다.

범위를 벗어난 GROUP BY position 에러

```
SELECT
dept_no, avg(sales_amount) FROM sales_tbl GROUP BY 3;
```

> MSGCAT_SEMANTIC_SORT_SPEC_RANGE_ERR

host variable를 그룹 조건으로 입력 불가

```
PREPARE A_stmt
FROM 'SELECT dept_no, avg(sales_amount) FROM sales_tbl GROUP BY ?';

EXECUTE A_stmt;
```

> MSGCAT_SEMANTIC_NO_GROUPBY_ALLOWED

WITH ROLLUP과 GROUPBY_NUM을 동시에 사용 못함

```
SELECT dept_no AS a1, name AS a2, avg(sales_amount) AS a3
FROM sales_tbl
WHERE sales_amount > 100
GROUP BY a1, a2 WITH ROLLUP HAVING GROUPBY_NUM() BETWEEN 1 AND 5
```

> MSGCAT_SEMANTIC_CANNOT_USE_GROUPBYNUM_WITH_ROLLUP

Aggregate/Analytic 함수의 인수 체크

COUNT(*), ROW_NUMBER(), RANK(), DENSE_RANK(), CUME_DIST(), PERCENT_RANK() 외에는 모두 function argument가 존재하므로 function 노드에 arg_list가 있어야 한다.

MEDIAN 함수와 OVER (order by...)

MEDIAN 함수는 OVER (order by...)를 지원하지 않는다.

```
SELECT col1, MEDIAN(col2) OVER (ORDER BY col1)
FROM tbl GROUP BY col1;
```

> MSGCAT_SEMANTIC_MEDIAN_FUNC_NOT_ALLOW_ORDER_BY

PARTITION BY NULL 제거

```
SELECT col1, MEDIAN(col2) OVER (PARTITION BY NULL) FROM tbl;

=> rewritten query
select tbl.col1, median(tbl.col2) over (order by 3) from tbl tbl
```

ORDER BY 관련

ORDER BY

```
SELECT ...
ORDER BY {col_name | expr | position} [ASC | DESC], ...]
[NULLS {FIRST | LAST}]
```

필요 없는 ORDER BY를 제거

```
select count(*) from athlete order by code

=> rewritten query
select count(*) from athlete athlete
```

position 의 범위가 select list 의 밖으로 나갔는지 확인

```
SELECT s_name from code order by 2;
```

> MSGCAT_SEMANTIC_SORT_SPEC_RANGE_ERR

Hierarchical Query 관련 (TBU)

CONNECT BY PRIOR <expr>에서
CONNECT_BY_ISLEAF과 CONNECT_BY_ISCYCLE을 사용 못하게 함
PT_LEVEL expr을 분석하여 cycle 체크를 할지 말지 결정함

Show statement 관련

SHOW에서 보여줄 meta data로 select_list를 대체하도록 rewrite한다.

Derived query 관련

Ambiguous reference 검사

derived query에서 AS attribute list가 동일한 이름이 존재하는지 검사한다.

```
SELECT SUM (n), AVG(n) FROM
(SELECT gold, silver FROM participant WHERE nation_code = 'KOR')
AS t(n, n);
```

> MSGCAT_SEMANTIC_AMBIGUOUS_REF_TO

Hidden column rewrite

Subquery Derived table에서 hidden column이 존재하는 경우 as_attr_list와 갯수를 맞춰주기 위해서 as_attr_list에 ha_<number>의 이름을 가진 임시 PT_NAME 노드들을 추가한다.

Expression 관련

Check CAST expression is well-formed

- COLLATE modifier가 있는 경우: CAST expression의 대상이 문자열인 경우 올바른 collation으로 변환하는지 확인한다.
- CAST Type: 목표 타입으로 변환이 가능한지 확인한다.

```
CAST (s2 AS STRING COLLATE utf8_bin)
CAST ('2008-12-25 10:30:20' AS TIMESTAMP)
```

> MSGCAT_SEMANTIC_COLLATE_NOT_ALLOWED
> MSGCAT_SEMANTIC_CS_MATCH_COLLATE
> MSGCAT_SEMANTIC_CANT_COERCE_TO
> MSGCAT_SEMANTIC_COERCE_UNSUPPORTED

💡 MSGCAT_SEMANTIC_CANT_COERCE_TO와 MSGCAT_SEMANTIC_COERCE_UNSUPPORTED의 차이

https://www.cubrid.org/manual/en/10.1/sql/function/typecast_fn.html의 CAST 룰을 참조

- 해당 룰이 표에 존재하지만 NO로 표기 => MSGCAT_SEMANTIC_CANT_COERCE_TO 반환
"Cannot coerce x to type y"

* 해당 룰이표에 존재하지 않음 =>
MSGCAT_SEMANTIC_COERCE_UNSUPPORTED 반환
"Casting x to type y is not supported"

LIMIT가 있는 경우 rewriting

LIMIT가 있는 경우 Aggregate를 derived table로 rewriting

```
select count(*) from athlete limit ?

=> rewritten query
select athlete.a_1 from
(select count(*) from athlete athlete) athlete (a_1)
where (inst_num()<= ?:0 )
```

LIMIT를 Numbering Expression으로 rewriting

LIMIT가 있을 때 다음의 네가지 경우에 따라 xxx_NUM 함수로 재작성된다.

1. LIMIT with ORDER BY => ORDERBY_NUM
2. LIMIT with GROUP BY => GROUPBY_NUM

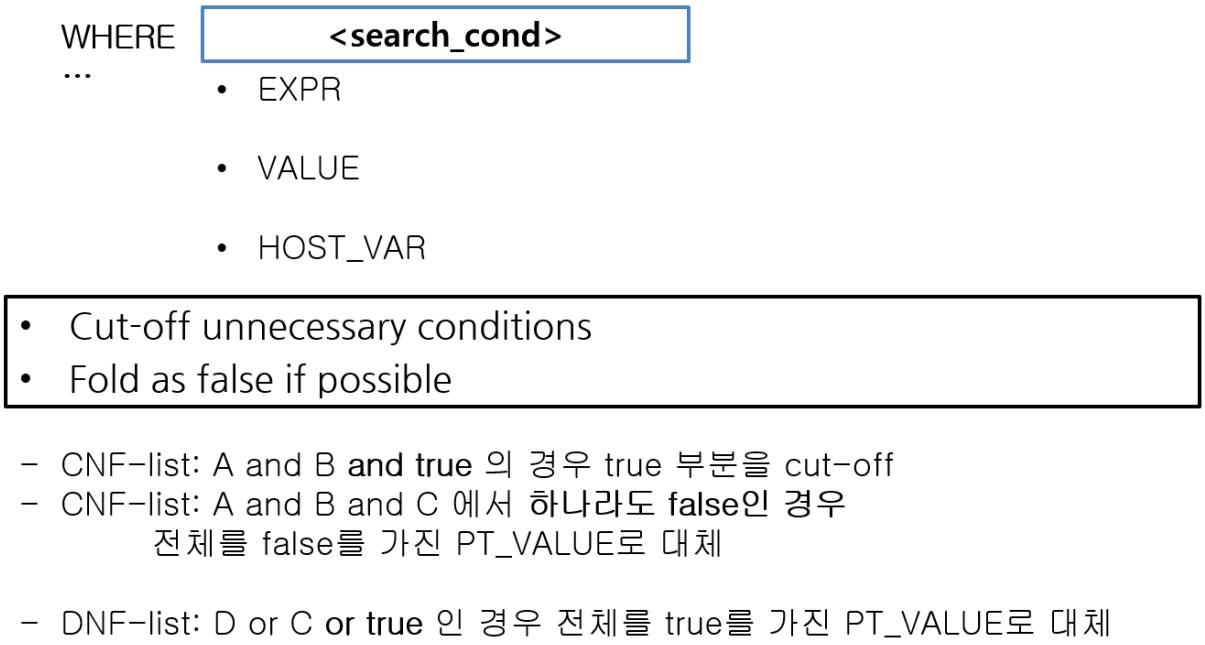
3. LIMIT with DISTINCT => ORDERBY_NUM
   (When a distinct query has neither orderby nor groupby clause)
4. LIMIT with WHERE => INST_NUM

```
1)
SELECT PERCENTILE_DISC(0.5) WITHIN GROUP
(ORDER BY math) AS pdisc FROM scores LIMIT 5;
=> rewritten query
... WITHIN GROUP
(ORDER BY math FOR ORDERBY_NUM() BETWEEN BETWEEN 1 AND 5) …

2)
SELECT deno, avg(sales_amount) FROM sales_tbl GROUP BY deno
LIMIT 5;
=> rewritten query
… FROM sales_tbl GROUP
BY deno HAVING GROUPBY_NUM() BETWEEN 1 AND 5;

3)
SELECT DISTINCT host_year from history LIMIT 3;
=> rewritten query
SELECT DISTINCT history.host_year from history history
for orderby_num()<= ?0;

4) SELECT * FROM code LIMIT 3;
=> rewritten query
SELECT code.s_name, code.f_name from code code
where (inst_num()<= ?0);
```

Search Condition 관련

WHERE, HAVING, ON Condition 등 search condition을 포함하는 clause에는 다음 그림의 rewriting을 수행한다.